

Programmation orientée objet - Héritage

L'héritage est le deuxième concept majeur de la programmation orientée objet que nous verrons dans le cours. Cela permet de créer des classes qui héritent des propriétés et des méthodes d'autres classes évitant ainsi la duplication de code.

Par exemple, une classe `Admin` peut hériter des propriétés et des méthodes de la classe `Utilisateur` qui est plus générale mais peut avoir des propriétés et des méthodes spécifiques à son rôle qui n'existent pas dans la classe parente.

Généralement, les classes parents contiendront plus de propriétés et de méthodes que les classes enfants qui elles ne contiendront que les cas d'exceptions.

Classes parentes et classes enfants

Une classe parent est une classe plus générale qui définit des propriétés et des méthodes communes à plusieurs classes enfants. Pour cela, on utilisera le mot-clé `extends` lors de la création de la classe enfant en spécifiant la classe parente.

```
class Utilisateur {
  constructor(nomUtilisateur, motDePasse) {
    this.nomUtilisateur = nomUtilisateur;
    this.motDePasse = motDePasse;
  }

  connecter() {
    console.log("Connecté en tant qu'utilisateur.");
  }
}

// La classe Admin hérite de la classe Utilisateur et de ses propriétés et de ses méthodes.
class Admin extends Utilisateur {
  constructor(nomUtilisateur, motDePasse) {
    super(nomUtilisateur, motDePasse);
    this.role = "admin";
  }

  retournerRole() {
    console.log(`Le rôle de l'admin ${this.nomUtilisateur} est ${this.role}.`);
  }
}

const admin = new Admin("Jean", "motdepasse");
const utilisateurOrdinaire = new Utilisateur("Marie", "motdepasse123");

utilisateurOrdinaire.connecter(); // Connecté en tant qu'utilisateur.
utilisateurOrdinaire.role; // undefined car le rôle n'est pas défini pour les utilisateurs ordinaires
```

```
admin.connecter(); // Connecté en tant qu'utilisateur.  
admin.retournerRole(); // Le rôle de l'admin Jean est admin.
```

Appel du constructeur parent

Pour avoir accès aux propriétés de la classe parente dans la classe enfant, il faut appeler le constructeur de la classe parente en utilisant le mot-clé `super`. Cela exécute le constructeur de la classe parente et initialise les propriétés héritées. Cela doit être fait avant d'utiliser `this` dans le constructeur de la classe enfant sinon une erreur sera levée.

On peut alors avoir accès aux propriétés héritées en utilisant `this`.

Classe parent et encapsulation

Attention, les classes privées ne sont pas héritées. Si une classe parente a des propriétés privées, elles ne seront pas accessibles dans la classe enfant. Vous devez alors définir des accesseurs (getters et setters) pour permettre l'accès aux propriétés privées de la classe parente.

Dans d'autres langages de programmation, on utiliserait des propriétés protégées pour permettre l'accès seulement aux sous-classes d'une propriété privée d'un parent. En JavaScript, la propriété protégée n'existe pas encore. La norme est alors de préfixer l'accesseur et le mutateur avec un underscore (`_`) mais la propriété est réellement publique.

```
class Utilisateur {  
    #nomUtilisateur;  
    #motDePasse;  
  
    constructor(nomUtilisateur, motDePasse) {  
        this.#nomUtilisateur = nomUtilisateur;  
        this.#motDePasse = motDePasse;  
    }  
  
    get _nomUtilisateur() {  
        return this.#nomUtilisateur;  
    }  
  
    set _nomUtilisateur(nouveauNom) {  
        this.#nomUtilisateur = nouveauNom;  
    }  
  
    get _motDePasse() {  
        return this.#motDePasse;  
    }  
  
    set _motDePasse(nouveauMotDePasse) {  
        this.#motDePasse = nouveauMotDePasse;  
    }  
}  
  
class Admin extends Utilisateur {
```

```

    #role;

    constructor(nomUtilisateur, motDePasse) {
        super(nomUtilisateur, motDePasse);
        this.#role = "admin";
    }

    get nomUtilisateur() {
        return this._nomUtilisateur;
    }

    get role() {
        return this.#role;
    }

    set role(nouveauRole) {
        this.#role = nouveauRole;
    }
}

```

Exemple concret

Dans l'exemple suivant, nous allons créer une classe pour les boutons de base du site et un bouton personnalisé pour déclencher une boîte modale d'erreur.

On utilisera l'héritage et l'encapsulation.

1. Créer une classe parente **Bouton** et définir les propriétés communes à tous les boutons.
2. Créer une classe enfant **BoutonErreur** qui hérite de la classe **Bouton** mais en ajoutant un message d'erreur.

```

class Bouton {
    #elementHTML;
    #conteneurHTML;
    #callback;
    #texte;

    constructor(texte, conteneurHTML, callback) {
        this.#texte = texte;
        this.#conteneurHTML = conteneurHTML;
        this.#elementHTML = null;
        this.#callback = callback;
        this.injecterHTML();
    }

    get _elementHTML() {
        return this.#elementHTML;
    }

    onClick() {
        this.#callback.call(this);
    }
}

```

```

injecterHTML() {
    let gabarit = `

```